

BCC202 - Estruturas de Dados I

Aula 04: Tipos Abstratos de Dados (TADs)

Pedro Silva

Universidade Federal de Ouro Preto – UFOP
Departamento de Computação – DECOM
Email: silvap@ufop.edu.br



Conteúdo

Definição de um Problema

Modularização

Tipo Abstrato de Dados (TAD)

Exemplo de uma TAD

Composição

Bibliotecas

Considerações Finais

Exercícios

Bibliografia

Definição de um Problema

Um problema de exemplo

Quero gerar um programa que calcula o perímetro de um terreno. Para isso, as informações de longitude e latitude de cada um dos quatro pontos do terreno são fornecidas. Esse ponto em $\mathbb{R}^2$ é fornecido na ordem das quatro esquinas.

Para resolver o problema, basta criarmos uma função para calcular a distância entre um ponto e outro e depois uma função para calcular o perímetro.

Um problema de exemplo

Quero gerar um programa que calcula o perímetro de um terreno. Para isso, as informações de longitude e latitude de cada um dos quatro pontos do terreno são fornecidas. Esse ponto em $\mathbb{R}^2$ é fornecido na ordem das quatro esquinas.

Para resolver o problema, basta criarmos uma função para calcular a distância entre um ponto e outro e depois uma função para calcular o perímetro.

No final, isso é um módulo que pode ser reutilizado em outros lugares.

Modularização

Mas o que é modularização?

Módulo é um “arquivo” com funções que representam **parte da implementação** de um programa.

- ▶ Funções **afins** são agrupadas por arquivos.
- ▶ Pode ser composto somente por funções criadas do zero, ou por **um** ou **mais módulos**.

Especificando novos módulos

E se pudéssemos criar módulos específicos de acordo com o domínio de cada programa que implementamos?

- ▶ O que precisaríamos implementar?
- ▶ Como cada módulo seria “ligado” ao *main()*?
- ▶ O que precisaríamos definir?
- ▶ Quais serão as vantagens de criar módulos de acordo com o domínio/contexto da aplicação?
- ▶ E que tal separar **especificação** da **implementação**? O que teríamos em cada um desses elementos?

Compilação ideal de vários módulos

No caso de um programa composto por **vários módulos**:

- ▶ Cada um dos módulos é compilado **separadamente**.
- ▶ Cada módulo compilado gera um arquivo objeto (**extensão .o ou .obj**)
- ▶ Após a compilação de todos os módulos, outra ferramenta, denominada **ligador (linker)**, é usada para juntar todos os arquivos em um único **arquivo executável**.
- ▶ Durante a ligação dos objetos, os códigos objetos das funções da biblioteca padrão de C também são incluídos (e.g., **stdio.h**¹).

Mas o que incluímos no arquivo com a função *main()*: **stdio.h** ou **stdio.c**? **Por quê?**

¹<http://www.cplusplus.com/reference/cstdio/>

Exemplo do porquê separar implementação da especificação

- Um programa para armazenar uma **lista** de **inteiros**.

Implementação por **Vetor**:

20	13	02	20
----	----	----	----

```
#include "lista.h"
void Insere(int x, Lista* L) {
    L->vetor[L->ultimo] = x;
    L->ultimo++;
} ...
```

Implementação por **Lista Encadeada**:



```
#include "lista.h"
void Insere(int x, Lista* L) {
    p = CriaNovaCelula(x);
    p->prox = L->primeiro->prox;
    L->primeiro->prox = p;
} ...
```

Programa do **usuário** de **Lista**.

```
#include "lista.h"

int main() {
    Lista* L;
    l = criaLista();
    int x;
    x = 20;
    Insere(x, L);

    .
    .
    .

    return 0;
}
```

Módulo e a lista do exemplo anterior

No exemplo anterior, as implementações não consideravam somente **funções afins**, mas há também dados envolvidos.

No exemplo, é feita uma **representação computacional** do que seria uma lista de números inteiros.

Temos uma **abstração** de uma lista de inteiros.
Criamos um Tipo Abstrato de Dados (TAD).

Tipo Abstrato de Dados (TAD)

Mas o que é um Tipo Abstrato de Dados (TAD)?

Definição

Abstração de uma “entidade” do mundo real para o mundo digital.

Especificação de um **conjunto de dados** mais **funções** que manipulam esses dados.

Em outras palavras

Especificação de um **tipo estruturado** mais **funções** para sua criação e manipulação.

Tipo **Abstrato** de Dados (TAD)

Abstração

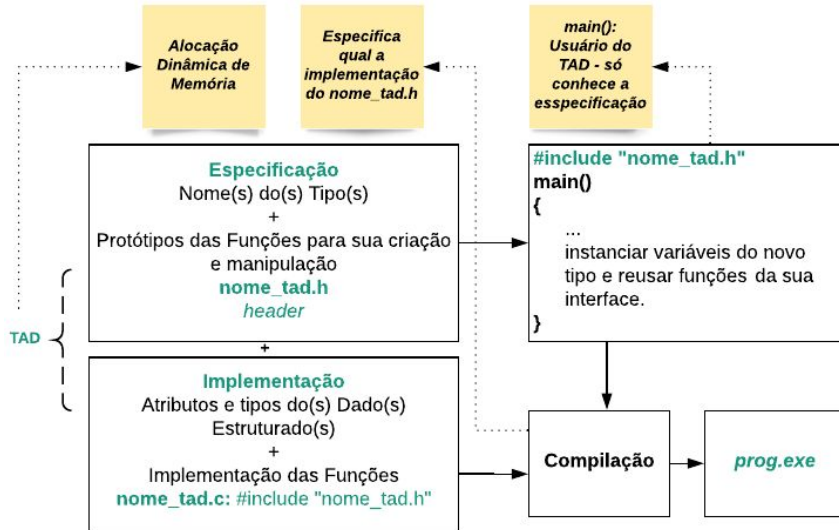
"É a habilidade de concentrar nos aspectos essenciais de um contexto qualquer, ignorando características menos importantes ou acidentais".²

Abstrato

"Abstraída a forma de implementação."

²<https://pt.wikipedia.org/wiki/Abstracao>

Ilustração dos principais elementos de um TAD.



Exemplo de uma TAD

Ponto no $\mathbb{R}^2$

Como nosso primeiro exemplo de TAD, vamos considerar a criação de um tipo de dado para representar um ponto no $\mathbb{R}^2$. Para isso, devemos definir um tipo abstrato, que denominaremos de *Ponto*, e o conjunto de funções que operam sobre esse tipo, descritas a seguir.

- ▶ *cria*: operação que aloca dinamicamente memória para um ponto com coordenadas x e y ;
- ▶ *libera*: operação que libera a memória alocada para um ponto;
- ▶ *acessa*: operação que devolve as coordenadas de um ponto;
- ▶ *atribui*: operação que atribui novos valores às coordenadas de um ponto;
- ▶ *distancia*: operação que calcula a distância entre dois pontos.

Ponto no $\mathbb{R}^2$: Definição da especificação do *Ponto* (*ponto.h*)

```
// TAD: Ponto (x,y)
#ifndef ponto_h
#define ponto_h

// Tipo exportado: somente o nome do NOVO TIPO
typedef struct ponto Ponto; // forward declaration

// Funções exportadas
Ponto* PontoCria(float x, float y);
Ponto* PontoLibera(Ponto* p);
void PontoAcessa(Ponto* p, float* x, float* y);
void PontoAtribui(Ponto* p, float x, float y);
float PontoDistancia(Ponto* p1, Ponto* p2);

#endif // !ponto_h
```

Ponto no $\mathbb{R}^2$: Implementação da especificação do *Ponto* (*ponto.c*)

```
#include <stdlib.h> // malloc, free, exit
#include <stdio.h>   // printf
#include <math.h>    // sqrt
#include "ponto.h"

// definição dos dados e seus tipos
struct ponto {
    float x;
    float y;
};
```

Ponto no $\mathbb{R}^2$: Implementação da especificação do *Ponto* (*ponto.c*)

```
Ponto* PontoCria (float x, float y) {
    Ponto* p = (Ponto*) malloc(sizeof(Ponto));
    if (p == NULL) {
        printf("Memória insuficiente!\n");
        exit(1);
        // return NULL;
    }
    p->x = x;
    p->y = y;
    return p;
}

Ponto* PontoLibera (Ponto* p) {
    free(p);
    p = NULL;
    return p;
}
```

Ponto no $\mathbb{R}^2$: Implementação da especificação do *Ponto* (*ponto.c*)

```
void PontoAcessa (Ponto* p, float* x, float* y) {
    *x = p->x;
    *y = p->y;
}

void PontoAtribui (Ponto* p, float x, float y) {
    p->x = x;
    p->y = y;
}

float PontoDistancia (Ponto* p1, Ponto* p2) {
    float dx = p2->x - p1->x;
    float dy = p2->y - p1->y;
    return sqrt(dx*dx + dy*dy);
}
```

Ponto no $\mathbb{R}^2$: Importando *Ponto* e suas funções (*main.c*)

```
#include <stdio.h>
#include "ponto.h" // incluindo a especificação

int main(){
    Ponto* p1 = PontoCria(2.0, 4.0); // alocando dinamicamente
    Ponto* p2 = PontoCria(8.0, 16.0);
    float x, y; // variáveis auxiliares
    PontoAcessa(p1, &x, &y); // manipulando o p1 instanciado
    printf("P1(%f, %f)\n", x, y);
    printf("Dist(P1 e P2): %f\n", PontoDistancia(p1, p2));
    PontoLibera(&p1);
    PontoLibera(&p2);
    return 0;
}
```

Ponto no $\mathbb{R}^2$: Compilando

Separadamente

```
» gcc -c ponto.c -Wall  
» gcc -c main.c -Wall  
» gcc -o exe ponto.o main.o
```

"Tudo junto"

```
» gcc -o exe main.c ponto.c -Wall
```

Ponto no $\mathbb{R}^2$: Compilando

Separadamente

```
» gcc -c ponto.c -Wall  
» gcc -c main.c -Wall  
» gcc -o exe ponto.o main.o
```

"Tudo junto" (não recomendado)

```
» gcc -o exe main.c ponto.c -Wall
```

Mas por que não é recomendado?

Ponto no $\mathbb{R}^2$: Compilando

Separadamente

```
» gcc -c ponto.c -Wall  
» gcc -c main.c -Wall  
» gcc -o exe ponto.o main.o
```

"Tudo junto"

```
» gcc -o exe main.c ponto.c -Wall
```

Mas por que não é recomendado?

Uma alteração em um dos arquivos força a compilação de todo o pacote.

Recapitulando

A seguir, algumas observações acerca do novo tipo, *Ponto*, implementado:

- ▶ A especificação (**ponto.h**) define apenas:
 - ▶ o **nome** do **novo tipo**
 - ▶ os **protótipos** das funções para:
 - ▶ **alocar** e **liberar** memória **dinamicamente**
 - ▶ **manipular** os dados de *Ponto* (pelos menos para ler e escrever (*get/set*))
- ▶ diretivas para pré-processamento (***#ifndef*** / ***#def*** / ***#endif***)
 - ▶ **Definição do mesmo .h duas vezes.**

Recapitulando

- ▶ A implementação (**ponto.c**):
 - ▶ inclui as bibliotecas padrões (e.g., para alocação dinamicamente de memória) e o **ponto.h**.
 - ▶ implementa **TODOS** os protótipos definidos em **ponto.h**.
 - ▶ pode implementar funções utilitárias, se necessário (não presentes no .h).

Recapitulando

- ▶ A implementação do *main* (**main.c**):
 - ▶ inclui as bibliotecas padrões (e.g., para entrada e saída de dados) e o **ponto.h**.
 - ▶ necessariamente declara variáveis como sendo **ponteiro** para o novo tipo **Ponto**.
 - ▶ não tem acesso aos campos de **Ponto**
 - ▶ os campos estão definidos no arquivo *ponto.c*
 - ▶ *main()* contém **#include "ponto.h"** (não há campos da(s) *struct(s)* no *ponto.h*)
 - ▶ toda **manipulação das variáveis do tipo *Ponto**** são realizadas via **funções declaradas no ponto.h**
 - ▶ deve sempre chamar as funções para **alocar e liberar memória dinamicamente**.

Composição

Principais Conceitos de Composição

- ▶ Um TAD pode ser reutilizado para definir outros novos tipos.
- ▶ Criar dependências entre módulos.
- ▶ Em outras palavras:
 - ▶ Um TAD pode reutilizar zero, um ou mais TADs.
 - ▶ structs que "contém" outro(s) struct(s).

Criar um **TAD Círculo** com as seguintes operações:

- ▶ *cria*: aloca memória dinamicamente para um círculo com centro (x,y) e raio r .
- ▶ *libera*: libera a memória alocada para um círculo.
- ▶ *area*: calcula a área do círculo.
- ▶ *interior*: verifica se um dado ponto está dentro do círculo.

Criar um **TAD Círculo** com as seguintes operações:

- ▶ *cria*: aloca memória dinamicamente para um círculo com centro (x,y) e raio r .
- ▶ *libera*: libera a memória alocada para um círculo.
- ▶ *area*: calcula a área do círculo.
- ▶ *interior*: verifica se um dado ponto está dentro do círculo.

Por onde começar? Há tipos abstratos de dados que podemos reutilizar?

Especificação do TAD *Circulo* (*circulo.h*)

```
// TAD Circulo
#ifndef circulo_h
#define circulo_h

#include <stdbool.h> // dependência de módulos
#include "ponto.h"   // tipo exportado/importado

typedef struct circulo Circulo; // forward declaration

#define PI 3.14159 // constante PI

// funções exportadas
Circulo* CirculoCria(Ponto *centro, float raio);
Circulo* CirculoLibera(Circulo *circ);
float CirculoArea(Circulo *circ);
bool CirculoInterior(Circulo *circ, Ponto *pt);

#endif // !circulo_h
```

Implementação do TAD *Circulo* (*circulo.c*)

```
#include <stdlib.h> // malloc, free, exit, NULL
#include <stdio.h>   // printf
#include "circulo.h"

// definição dados e seus tipos
struct circulo{
    Ponto* centro;
    float raio;
};
```

Implementação do TAD *Circulo* (*circulo.c*)

```
// alternativa: receber (x,y) e raio: instanciar o ponto
Circulo* CirculoCria(Ponto *centro, float raio) {
    Circulo* circ = (Circulo*) malloc (sizeof(Circulo));
    if (circ == NULL) {
        printf("Memória insuficiente!\n");
        exit(1);
    }
    circ->centro = centro;
    circ->raio = raio;
    return circ;
}

Circulo* CirculoLibera(Circulo *circ){
    PontoLibera(circ->centro); // primeiro libera a(s) parte(s)
    free(circ); // depois libera o todo
    circ = NULL;
    return circ;
}
```

Implementação do TAD *Circulo* (*circulo.c*)

```
float CirculoArea(Circulo *circ) {  
    return PI * circ->raio * circ->raio;  
}  
  
bool CirculoInterior(Circulo *circ, Ponto *pt) {  
    float d = PontoDistancia(circ->centro, pt);  
    return (d < (circ->raio));  
}
```

Importando o TAD *Circulo* e suas funções (*main_circulo.c*)

```
#include <stdio.h>
#include "circulo.h" //basta incluir circulo.h (contém ponto.h)

int main(){
    // instanciando e manipulando variáveis do tipo Ponto
    Ponto* p1 = PontoCria(8.0, 15.0);
    Ponto* p2 = PontoCria(8.0, 16.0);
    // instanciando e manipulando variáveis do tipo Circulo
    Circulo* circ = CirculoCria(p2, 4.9);
    printf("Area(cir): %f\n", CirculoArea(circ));
    if(CirculoInterior(circ, p1))
        printf("P1 está em Circ\n");
    circ = CirculoLibera(circ); // liberando memória alocada
    p1 = PontoLibera(p1);
    // p2 já foi liberado ao liberar o circ
    return 0;
}
```

Separadamente

```
» gcc -c ponto.c -Wall  
» gcc -c circulo.c -Wall  
» gcc -c main_circ.c -Wall  
» gcc -o exe ponto.o circulo.o main_circ.o
```

Recapitulando

A seguir, algumas observações acerca do novo tipo, *Circulo*, implementado:

- ▶ `circulo.h` inclui `ponto.h`: há dependência de módulos
- ▶ `circulo.c` não tem acesso aos dados de *Ponto*, logo precisa reutilizar as funções para manipular variáveis do tipo `*Ponto` (ponteiro de `Ponto`) que estão em `ponto.h`
- ▶ `main_circ.c` não tem acesso aos dados de *Ponto* e de *Circulo*, logo precisa utilizar as funções para manipulações presentes em `ponto.h` e `circulo.h`

Bibliotecas

Em algumas situações, não queremos definir novos tipos, mas apenas agrupar funções afins.

- ▶ Exemplo: para definir uma **biblioteca** com **métodos de ordenação**, teríamos.
 - ▶ **ordenacao.h**: conteria apenas os protótipos dos métodos de ordenação, sem definir novos tipos.
 - ▶ **ordenacao.c**: conteria apenas as implementações dos protótipos.
 - ▶ **main.c** (ou outro módulo dependente): teria o `#include "ordenacao.h"`

Considerações Finais

Motivação para Definição de TADs

- ▶ A ideia central é **encapsular** ou **esconder** de quem usa determinado tipo a forma concreta como foi implementado.
- ▶ Com isso, **desacoplamos** a **implementação** do **uso**:
 - ▶ Facilitando a **manutenção** e aumentando o potencial de **reutilização** do tipo criado.
- ▶ Agrupar tipos e funções com funcionalidades relacionadas - **alta coesão**.
- ▶ Usuário, *main()* ou mesmo outro módulo, só "enxerga" a interface, não a implementação - **baixo acoplamento**.

Vantagens da modularização

- ▶ Implementar um programa em vários módulos e estabelecer dependência entre eles favorece:
 - ▶ manutenção;
 - ▶ reúso;
 - ▶ corretude;
 - ▶ legibilidade;
 - ▶ encapsulamento.

TAD vs Bibliotecas

- ▶ Exemplos de módulos:
 - ▶ TAD: novo(s) tipo(s) mais funções para manipulá-los - ao menos: criar, liberar, ler/escrever seus dados.
 - ▶ Biblioteca: funções afins
- ▶ Usuários, *main()* ou outro(s) módulo(s), incluem a especificação do módulo.

Próxima Aula

Recursividade.

Exercícios

Exercício 1

Sobre o *TAD Circulo*, especifique e implemente funções para ler e atualizar os valores do ponto central e raio de um círculo (*get/set*).

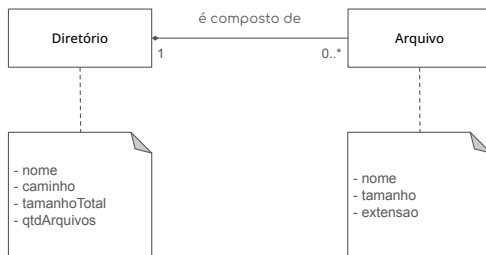
Exercício 2

Considerando o *TAD Ponto*, implemente o *TAD Quadrado*. Seu TAD deve conter:

- ▶ *cria*: operação que aloca dinamicamente memória para dois pontos com coordenadas x e y : (x_{min}, y_{min}) e (x_{max}, y_{max}) ;
- ▶ *libera*: operação que libera a memória alocada para os pontos do quadrado;
- ▶ *acessa*: operação que devolve as coordenadas dos dois pontos;
- ▶ *atribui*: operação que atribui novos valores às coordenadas dos dois pontos;
- ▶ *sobreposicao*: operação que retorna se dois quadrados se sobrepõe (1) ou não (0).

Exercício 3

Um diretório, ou pasta, tem os seguintes atributos, ou características: *i)* nome, *ii)* caminho, *iii)* tamanho total, que é a soma dos tamanhos dos arquivos nele contido, e *iv)* quantidade de arquivos. O diretório pode conter zero ou mais arquivos (no máximo 50). Já um arquivo, tem os seguintes atributos: *i)* nome, *ii)* tamanho e *iii)* extensão. Considere que o tamanho de cada um dos arquivos, e também de cada diretório, é representando por um número real, sendo a unidade correspondente megabyte (MB). Essa composição pode ser expressa diagramaticamente da seguinte forma:



Exercício 3

Assim sendo, sua solução **DEVE** contemplar, além dos novos tipos de dados, as especificações e implementações das funções a seguir:

- ▶ ler e atualizar (*i.e.*, *get/set*) nome e extensão de um arquivo;
- ▶ ler e atualizar (*i.e.*, *get/set*) nome e caminho de um diretório;
- ▶ alocar e liberar memória dinamicamente;
- ▶ retornar o tamanho total de um diretório;
- ▶ retornar a quantidade de arquivos presentes em um diretório;
- ▶ adicionar um arquivo em um diretório;
- ▶ remover um arquivo de um diretório.

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados do livro-texto indicado a seguir:



Celes, Waldemar and Cerqueira, Renato and Rangel, José

Introdução a Estruturas de Dados com Técnicas de Programação em C.
Elsevier Brasil, 2016. ISBN 978-85-352-8345-7.



Anany Levitin.

Introduction To Design And Analysis Of Algorithms.
3rd Edition, 2012.



T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.

Introduction to Algorithms.

The MIT Press, 3rd edition, 2009. ISBN-13: 978-0-262-53305-8.